

TD04 : Architecture MVC (Modèle, Vue, Contrôleur)

Table des matières

1) Problématique.....	1
2) Exercices	1
3) Conclusion.....	7

1) Problématique

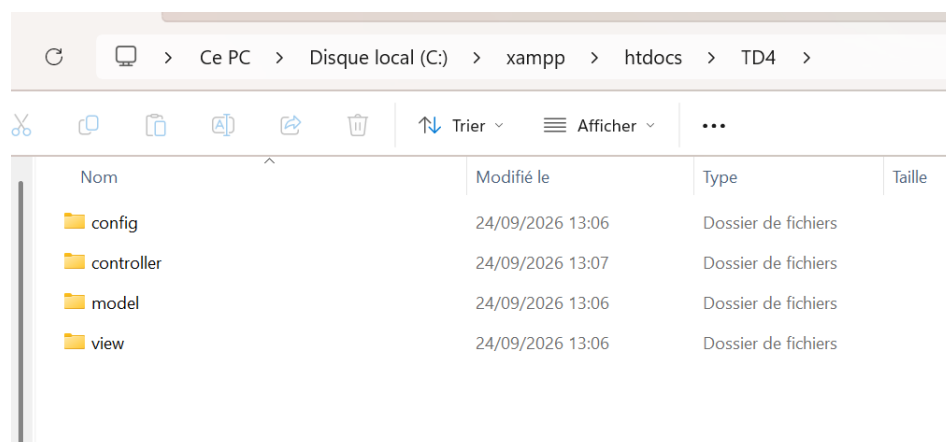
Nous souhaitons apprendre à bien organiser notre code grâce au design pattern MVC (Modèle - Vue - Contrôleur). Pour cela nous allons effectuer plusieurs exercices afin de séparer notre site web en trois parties distinctes (le Modèle pour la gestion des données, la Vue pour l'affichage en HTML et le Contrôleur avec le Routeur pour gérer les actions) en s'aidant d'outils comme Xampp et Notepad++.

2) Exercices

Exercice 1 : Le modèle

1) Création des dossiers

On commence par créer les dossiers dans notre dossier TD4 : config, controller, model, view et view/voiture.



2) Modification de ModelVoiture.php

On renomme notre fichier Voiture.php en ModelVoiture.php. Dans le fichier, on renomme aussi la classe Voiture en ModelVoiture. On enlève la fonction afficher() car maintenant c'est la vue qui va s'occuper de l'affichage. On change aussi le setFetchMode pour qu'il fabrique bien des objets ModelVoiture.

```
<?php
require_once "Model.php";

class ModelVoiture {
    private $immatriculation;
    private $marque;
    private $couleur;

    public function __construct($m = NULL, $c = NULL, $i = NULL) {
        if (!is_null($m) && !is_null($c) && !is_null($i)) {
            $this->marque = $m;
            $this->couleur = $c;
            $this->immatriculation = $i;
        }
    }

    public function getImmatriculation() {
        return $this->immatriculation;
    }

    public function getMarque() {
        return $this->marque;
    }

    public function getCouleur() {
        return $this->couleur;
    }

    public function setImmatriculation($immatriculation) {
        $this->immatriculation = $immatriculation;
    }

    public function setMarque($marque) {
        $this->marque = $marque;
    }

    public function setCouleur($couleur) {
        $this->couleur = $couleur;
    }

    public static function getAllVoitures() {
        $rep = Model::$pdo->query("SELECT * FROM voiture");
        $rep->setFetchMode(PDO::FETCH_CLASS, 'ModelVoiture');
        $stab_voit = $rep->fetchAll();
        return $stab_voit;
    }

    public static function getVoitureByImmat($immat) {
        $sql = "SELECT * from voiture WHERE immatriculation = :nom_tag";
        $req_prep = Model::$pdo->prepare($sql);
        $values = array(
            "nom_tag" => $immat
        );
        $req_prep->execute($values);
        $req_prep->setFetchMode(PDO::FETCH_CLASS, 'ModelVoiture');
        $stab_voit = $req_prep->fetchAll();

        if (empty($stab_voit)) {
            return false;
        }
    }
}
```

3) Déplacement des fichiers

On met les fichiers ModelVoiture.php et Model.php dans le dossier model/, et on met Conf.php dans le dossier config/ :

Ce PC > Disque local (C:) > xampp > htdocs > TD4 > model				
Nom	Modifié le	Type	Taille	
Model.php	24/09/2026 13:06	Fichier PHP	1 Ko	
ModelVoiture.php	24/09/2026 13:06	Fichier PHP	2 Ko	

Ce PC > Disque local (C:) > xampp > htdocs > TD4 > config				
Nom	Modifié le	Type	Taille	
Conf.php	24/09/2026 13:06	Fichier PHP	1 Ko	

4) Correction du chemin dans Model.php

Comme on a déplacé les fichiers, on doit corriger le chemin dans Model.php pour qu'il retrouve Conf.php dans le dossier config.

```
<?php
require_once __DIR__ . '/../config/Conf.php';
```

Exercice 2 : La vue

Création de list.php

On crée le fichier list.php dans view/voiture/. Ce fichier contient juste du code HTML avec une petite boucle PHP pour afficher les voitures qu'on lui donne dans la variable \$tab_v.

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="UTF-8">
5 <title>Liste des voitures</title>
6 </head>
7 <body>
8 <?php
9     foreach ($tab_v as $v) {
10         $immat = $v->getImmatriculation();
11         echo '<p> Voiture d\'immatriculation <a href="routeur.php?action=read&immat=' . $immat . '"> . $immat . '</a>.</p>';
12     }
13 <?>
14 </body>
15 </html>
16
```

Exercice 3 : le contrôleur

1) Création de ControllerVoiture.php

On crée le fichier ControllerVoiture.php dans le dossier controller/. Ce fichier sert d'intermédiaire : il appelle ModelVoiture pour récupérer les voitures dans \$tab_v, puis il appelle la vue list.php pour les afficher à l'écran.

```
<?php
require_once ('../model/ModelVoiture.php');

class ControllerVoiture {
    public static function readAll() {
        $tab_v = ModelVoiture::getAllVoitures();
        require ('../view/voiture/list.php');
    }

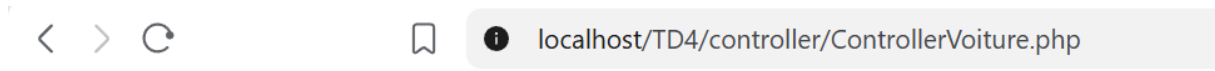
    public static function read() {
        $immat = $_GET['immat'];
        $v = ModelVoiture::getVoitureByImmat($immat);
        if ($v == false) {
            require ('../view/voiture/error.php');
        } else {
            require ('../view/voiture/detail.php');
        }
    }

    public static function create() {
        require ('../view/voiture/create.php');
    }

    public static function created() {
        $v = new ModelVoiture($_GET['marque'], $_GET['couleur'], $_GET['immat']);
        $v->save();
        self::readAll();
    }
}
?>
```

2) Test dans le navigateur

On va sur la page <http://localhost/TD4/controller/ControllerVoiture.php> pour tester si les voitures s'affichent bien :



Voiture d'immatriculation [AA000AA.](#)

Voiture d'immatriculation [BB111BB.](#)

Exercice 4 : Le routeur

1) Création de la classe et du routeur

On modifie ControllerVoiture.php pour ranger notre code dans une classe avec une fonction readAll(). Ensuite on crée un fichier routeur.php dans controller/ qui appelle directement cette fonction.

```
<?php
require_once 'ControllerVoiture.php';
ControllerVoiture::readAll();
?>
```

Exercice 5 : Un vrai routeur

1) Récupérer l'action dans l'adresse

On modifie routeur.php pour qu'il lise l'action passée dans l'adresse du navigateur grâce à \$_GET['action'], et qu'il lance automatiquement la fonction correspondante.

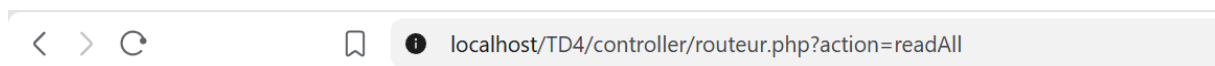
```
<?php
require_once 'ControllerVoiture.php';

$action = $_GET['action'];
ControllerVoiture::$action();
?>
```

2) Test dans le navigateur

On teste en ajoutant ?action=readAll à la fin de l'adresse :

<http://localhost/TD4/controller/routeur.php?action=readAll>



Voiture d'immatriculation [AA000AA.](#)

Voiture d'immatriculation [BB111BB.](#)

Exercice 6 : Voir le détail d'une voiture et gérer les erreurs

1) Création de detail.php

On crée le fichier detail.php dans view/voiture/ pour afficher toutes les infos d'une seule voiture (immatriculation, marque et couleur).

```

<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Détail de la voiture</title>
  </head>
  <body>
    <?php
    echo 'Voiture ' . $v->getImmatriculation() . ' de marque ' . $v->getMarque() . ' (couleur : ' . $v->getCouleur() . ')</p>';
    ?>
  </body>
</html>

```

2) Ajout de la fonction read

Dans ControllerVoiture.php, on ajoute la fonction read() qui prend l'immatriculation dans l'adresse (\$_GET['immat']), cherche la voiture avec getVoitureByImmat, et envoie le résultat à detail.php. On teste avec

<http://localhost/TD4/controller/routeur.php?action=read&immat=AA000AA> :



```

<?php
require_once ('../model/ModelVoiture.php');

class ControllerVoiture {
    public static function readAll() {
        $tab_v = ModelVoiture::getAllVoitures();
        require ('../view/voiture/list.php');
    }

    public static function read() {
        $immat = $_GET['immat'];
        $v = ModelVoiture::getVoitureByImmat($immat);
        if ($v == false) {
            require ('../view/voiture/error.php');
        } else {
            require ('../view/voiture/detail.php');
        }
    }
}

```

Voiture AA000AA de marque Toyota (couleur : Rouge)

3) Liens cliquables sur la liste

Dans list.php, on ajoute un lien <a> sur l'immatriculation pour pouvoir cliquer dessus et aller directement sur sa page de détail.

n [AA000AA](#).

n [BB111BB](#).

4) Page d'erreur error.php

On crée une vue error.php dans view/voiture/ qui affiche un message d'erreur si la voiture n'existe pas. Dans la fonction read(), si aucune voiture n'est trouvée, on ouvre cette page d'erreur. On teste en mettant une fausse immatriculation dans l'adresse :

```

<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Erreur</title>
  </head>
  <body>
    <p>Erreur : aucune voiture ne correspond à cette immatriculation.</p>
  </body>
</html>

```

Exercice 7 : Ajouter une voiture avec un formulaire

1) Création du formulaire create.php

On crée le fichier create.php dans view/voiture/ qui contient le formulaire pour taper les infos de la voiture. On met un champ caché `<input type='hidden' name='action' value='created'>` pour dire au routeur quoi faire quand on clique sur Envoyer. Dans ControllerVoiture.php, on ajoute la fonction create() pour afficher cette page.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Créer une Voiture</title>
</head>
<body>
  <form method="get" action="routeur.php">
    <fieldset>
      <legend>Nouvelle Voiture :</legend>
      <input type="hidden" name="action" value="created">
      <p>
        <label for="immat_id">Immatriculation :</label>
        <input type="text" placeholder="ex: 256AB34" name="immat" id="immat_id" required/>
      </p>
      <p>
        <label for="marque_id">Marque :</label>
        <input type="text" placeholder="ex: Toyota" name="marque" id="marque_id" required/>
      </p>
      <p>
        <label for="couleur_id">Couleur :</label>
        <input type="text" placeholder="ex: Bleu" name="couleur" id="couleur_id" required/>
      </p>
      <p>
        <input type="submit" value="Envoyer" />
      </p>
    </fieldset>
  </form>
</body>
</html>
```

2) Enregistrement avec created

Dans ControllerVoiture.php, on ajoute la fonction created() qui récupère ce qu'on a tapé dans le formulaire, crée une nouvelle voiture, l'enregistre dans la base avec save(), et rappelle readAll() pour réafficher la liste avec la nouvelle voiture ajoutée.

```
public static function create() {
    require ('../view/voiture/create.php');
}

public static function created() {
    $v = new ModelVoiture($_GET['marque'], $_GET['couleur'], $_GET['immat']);
    $v->save();
    self::readAll();
}
?>
```

3) Test d'ajout d'une voiture

On remplit le formulaire et on clique sur Envoyer. La voiture est bien enregistrée dans MySQL et on revient sur la liste où la nouvelle voiture apparaît bien.

Nouvelle Voiture : _____

Immatriculation :

Marque :

Couleur :

Voiture d'immatriculation [abcdefg](#).

3) Conclusion

En conclusion, ce TD nous a permis de bien comprendre comment fonctionne l'architecture MVC et comment séparer notre code pour qu'il soit plus propre. Nous avons bien compris le rôle de chaque élément : le Modèle pour la base de données, la Vue pour l'affichage en HTML, et le Contrôleur avec le Routeur pour gérer les actions demandées par l'utilisateur.